Ponguru Bhanu

cs23btech11046@iith.ac.in | bhanuponguru@gmail.com | +91 7416972968 | LinkedIn | GitHub

EDUCATION

Institution	Degree / Board	Passing Year	Score / CGPA
Indian Institute of Technology Hyderabad	B.Tech	2027	7.60
Devnar School for the Blind	Class XII (Board of Secondary Education, Telangana State)	2021	9.70%
Sri Chaithanya	Class X (Telangana State Board of Intermediate Education)	2023	97.10%

SKILLS

Programming: Python, C, C++, JavaScript, Rust (basic)

Machine Learning: Deep Learning, NLP, Computer vision, CNN, RNN, Transformers, LLMs, PyTorch, scikit-learn

Backend & Systems: FastAPI, Flask, Django, REST APIs, Linux, Nginx, Docker, Compilers

Databases: SQL based databases, MongoDB

Tools: Git, Bash, Jupyter, ONNX, CI/CD, React, React Native, MLFlow

CERTIFICATIONS

AI Engineer for Developers Associate -- DataCamp | Oct 2026

PROJECTS

Custom Transformer Decoder

Kaggle

- Implemented a 62M-parameter GPT-style Transformer decoder achieving a validation perplexity of 18.75 after training on 0.7B tokens with custom multi-head attention and Rotary Position Embeddings (RoPE).
- Engineered a GPU training pipeline with custom data loading and batching, mixed-precision autocasting, and memory-efficient tensor handling.
- **Technologies used:** Python, PyTorch

Uncertainty-Aware 2D Pose Transition (Ongoing Research Project)

- Working on human pose transition, which transforms a set of 2D skeletal keypoints from one pose to another while preserving anatomical constraints.
- Contributing to a research project that formulates uncertainty in pose representations arising from factors such as occlusion and image blur.
- Extending the pose-transition formulation introduced in Dittakavi et al., "Pose-to-Pose: A New Task and Benchmark for Human Pose Transition in Yoga" .

OpenJarvis CLI

GithubRepository

- Implemented a LangGraph-based agent orchestrator that routes requests from a generalist to configurable domain specialists, supports specialist-to-specialist delegation, and enforces a configurable hop limit to prevent delegation loops.
- Implemented specialist-scoped tool access with a two-phase retrieval pipeline: tool descriptions are embedded and

ranked by cosine similarity, while retrieval and execution are restricted to the tools permitted for the active specialist.
- Built 49 built-in tools across 9 modules covering file operations, Git, web/API access, code execution, data processing, mathematics, date/time, and persistent memory, with runtime permission checks and argument-level constraints before tool execution.
- **Technologies used:** Python, LangGraph, LangChain, FastEmbed, PyYAML, PyInstaller